

Travaux pratiques - Créer un exemple d'application Web dans un conteneur Docker

Objectifs

Partie 1 : Lancer la DEVASC VM

Partie 2 : Créer un script Bash simple

Partie 3 : Créer un exemple d'application Web

Partie 4 : Configurer l'application Web pour utiliser les fichiers de site Web

Partie 5 : Créer un script Bash pour créer et exécuter un conteneur Docker

Partie 6 : Construire, exécuter et vérifier le conteneur Docker

Contexte/scénario

Dans ce TP, vous allez passer en revue les techniques de script bash de base car le script bash est une condition préalable pour le reste du laboratoire. Vous allez ensuite créer et modifier un script Python pour une application web simple. Ensuite, vous allez créer un script bash pour automatiser le processus de création d'un fichier Dockerfile, de construction du conteneur Docker et d'exécution du conteneur Docker. Enfin, vous utiliserez les commandes **docker** pour étudier les subtilités de l'instance de conteneur Docker.

Ressources requises

- 1 PC avec système d'exploitation de votre choix
- Boîte virtuelle ou VMWare
- Machine virtuelle DEVASC

Instructions

Partie 1 : Lancez la machine virtuelle DEVASC

Si vous n'avez pas encore terminé **TP - Installer l'environnement de laboratoire de machines virtuelles**, faites-le maintenant. Si vous avez déjà terminé ce laboratoire, lancez la machine virtuelle DEVASC maintenant.

Partie 2 : Créez un script de bash simple

Les connaissances Bash sont essentielles pour travailler avec l'intégration continue, le déploiement continu, les conteneurs et avec votre environnement de développement. Les scripts Bash aident les programmeurs à automatiser diverses tâches dans un seul fichier de script. Dans cette partie, vous allez examiner brièvement comment créer un script bash. Plus tard dans le laboratoire, vous utiliserez un script bash pour automatiser la création d'une application Web à l'intérieur d'un conteneur Docker.

Étape 1: Créez un fichier de script bash vide.

Changez votre répertoire de travail en `~/labs/devnet-src/sample-app` et ajoutez un nouveau fichier appelé `user-input.sh`.

Étape 2: Ouvrez le fichier dans l'éditeur de texte nano.

Utilisez la commande **nano** pour ouvrir l'éditeur de texte nano.

Étape 3: Ajoutez le 'she-bang' en haut du script.

De là, vous pouvez entrer des commandes pour votre script bash. Utilisez les touches fléchées pour naviguer en **nano**. Notez les commandes en bas (non illustrées ici) pour gérer le fichier. Le symbole carat (^) indique que vous utilisez la touche CTRL ou Commande de votre clavier. Par exemple, pour quitter **nano**, tapez CTRL+X.

Ajoutez le 'she-bang' qui indique au système que ce fichier inclut des commandes qui doivent être exécutées dans le shell bash.

```
#!/bin/bash
```

Remarque : Vous pouvez utiliser un éditeur de texte graphique ou ouvrir le fichier avec VS Code. Cependant, vous devriez être familier avec les éditeurs de texte en ligne de commande comme **nano** et **vim**. Recherchez sur Internet des didacticiels pour rafraîchir vos compétences ou en apprendre davantage à leur sujet.

Étape 4: Ajoutez des commandes bash simples au script.

Entrez quelques commandes bash simples pour votre script. Les commandes suivantes demanderont à l'utilisateur un nom, définiront le nom sur une variable appelée **UserName** et afficheront une chaîne de texte avec le nom de l'utilisateur.

```
echo -n "Enter Your Name: "  
read userName  
echo "Your name is $userName."
```

Étape 5: Quittez nano et enregistrez votre script.

Appuyez sur **CTRL+X**, puis sur **Y**, puis sur **ENTER** pour quitter **nano** et enregistrer votre script.

Étape 6: Exécutez votre script à partir de la ligne de commande.

Vous pouvez l'exécuter directement à partir de la ligne de commande à l'aide de la commande suivante.

```
bash user-input.sh
```

Étape 7: Modifiez le mode du script en fichier exécutable pour tous les utilisateurs.

Changez le mode du script en exécutable à l'aide de la commande **chmod**. Définissez les options sur **a+x** pour rendre le script exécutable (x) par tous les utilisateurs (a). Après avoir utilisé **chmod**, les autorisations de notification ont été modifiées pour les utilisateurs, les groupes et autres afin d'inclure le "x" (exécutable).

Étape 8: Renommez le fichier pour supprimer l'extension .sh.

Vous pouvez renommer le fichier pour supprimer l'extension afin que les utilisateurs n'aient pas à ajouter .sh à la commande pour exécuter le script.

Étape 9: Exécutez le script à partir de la ligne de commande.

Maintenant, le script peut être exécuté à partir de la ligne de commande sans la commande **source** ou une extension. Pour exécuter un script bash sans la commande source, vous devez préfacier le script avec **./**.

Étape 10: Enquêtez d'autres scripts bash.

Si vous avez peu ou pas d'expérience dans la création de scripts bash, prenez le temps de rechercher sur Internet des tutoriels bash, des exemples bash et des jeux bash.

Partie 3 : Créer un exemple d'application Web

Avant de pouvoir lancer une application dans un conteneur Docker, nous devons d'abord avoir l'application. Dans cette partie, vous allez créer un script Python très simple qui affichera l'adresse IP du client lorsque le client visite la page Web.

Étape 1: Installez Flask et ouvrez un port sur le pare-feu VM DEVASC.

Les développeurs d'applications Web utilisant Python utilisent généralement un framework. Un framework est une bibliothèque de code pour faciliter la création d'applications Web fiables, évolutives et maintenables pour les développeurs. Flask est un framework d'application web écrit en Python. D'autres cadres incluent Tornado et Pyramid.

Vous allez utiliser cette infrastructure pour créer l'exemple d'application Web. Flask reçoit des demandes, puis fournit une réponse à l'utilisateur dans l'application Web. Ceci est utile pour les applications Web dynamiques car il permet l'interaction utilisateur et le contenu dynamique. Ce qui rend votre exemple d'application Web dynamique, c'est qu'elle affichera l'adresse IP du client.

Remarque : La compréhension des fonctions, des méthodes et des bibliothèques de Flask dépasse le cadre de ce cours. Il est utilisé dans ce laboratoire pour montrer à quelle vitesse vous pouvez mettre en service une application Web. Si vous voulez en savoir plus, recherchez sur Internet plus d'informations et didacticiels sur le framework Flask.

Ouvrez une fenêtre de terminal et importez le **flask**.

```
pip3 install flask
```

Étape 2: Ouvrez le fichier `sample_app.py`.

Ouvrez le fichier **sample_app.py** situé dans le répertoire `~/labs/devnet-src/sample-app`. Vous pouvez le faire dans VS Code ou vous pouvez utiliser un éditeur de texte en ligne de commande comme **nano** ou **vim**.

Étape 3: Ajoutez les commandes pour importer des méthodes à partir de flask.

Ajoutez les commandes suivantes pour importer les méthodes requises à partir de la bibliothèque de flask.

```
from flask import Flask
from flask import request
```

Étape 4: Créez une instance de la classe Flask.

Créez une instance de la classe Flask et nommez-la **sample**. Assurez-vous d'utiliser deux traits de soulignement avant et après le "nom".

```
sample = Flask(__name__)
```

Étape 5: Définissez une méthode pour afficher l'adresse IP du client.

Ensuite, configurez Flask de sorte que lorsqu'un utilisateur visite la page par défaut (répertoire racine), il affiche un message avec l'adresse IP du client.

```
@sample.route("/")
def main():
    return "You are calling me from " + request.remote_addr + "\n"
```

Remarquez l'instruction `@sample.route("/")` Flask. Les cadres de travail tels que Flask utilisent une technique de routage (`.route`) pour faire référence à une URL d'application (ceci à ne pas confondre avec le routage réseau). Ici, le "/" (répertoire racine) est lié à la fonction `main()`. Ainsi, lorsque l'utilisateur va à `http://localhost:8080/` (répertoire racine) URL, la sortie de l'instruction `return` sera affichée dans le navigateur.

Étape 6: Configurez l'application pour qu'elle s'exécute localement.

Enfin, configurez Flask pour exécuter l'application localement à l'adresse `http://0.0.0.0:8080`, qui est également `http://localhost:8080`. Assurez-vous d'utiliser deux traits de soulignement avant et après "name", et avant et après "main".

```
if __name__ == "__main__":
    sample.run (host="0.0.0.0", port=8080)
```

Étape 7: Enregistrez et exécutez votre exemple d'application Web.

Enregistrez votre script et exécutez-le à partir de la ligne de commande. Vous devriez voir la sortie suivante qui indique que votre serveur "sample-app" est en cours d'exécution. Si vous ne voyez pas la sortie suivante ou si vous recevez un message d'erreur, vérifiez attentivement votre script `sample_app.py`.

```
devasc@labvm:~/labs/devnet-src/sample-app$ python3 sample_app.py
* Serving Flask app "sample-app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

Étape 8: Vérifiez que le serveur fonctionne.

Vous pouvez vérifier que le serveur fonctionne de deux façons.

- Ouvrez le navigateur Web Chromium et entrez `0.0.0.0:8080` dans le champ URL. Vous devriez obtenir la sortie suivante:

You are calling me from 127.0.0.1

Si vous recevez une réponse "HTTP 400 Bad Request", vérifiez attentivement votre script `sample_app.py`.

- Ouvrez une autre fenêtre de terminal et utilisez l'outil URL de ligne de commande (cURL) pour vérifier la réponse du serveur.

```
curl http://0.0.0.0:8080
```

Étape 9: Arrêtez le serveur.

Retournez à la fenêtre du terminal où le serveur est en cours d'exécution et appuyez sur CTRL+C pour arrêter le serveur.

Partie 4 : Configurer l'application Web pour utiliser les fichiers de site Web

Dans cette partie, créez l'exemple d'application Web pour inclure une page `index.html` et une spécification `style.css`. Le `index.html` est généralement la première page chargée dans le navigateur Web d'un client lors de la visite de votre site Web. Le `fichier style.css` est une feuille de style utilisée pour personnaliser l'apparence de la page Web.

Étape 1: Explorez les répertoires qui seront utilisés par l'application web.

Les **modèles** de répertoires et **statiques** sont déjà dans le répertoire `sample-app`. Ouvrez `index.html` et `style.css` pour afficher leur contenu. Si vous êtes familier avec HTML et CSS, n'hésitez pas à personnaliser ces répertoires et fichiers autant que vous le souhaitez. Cependant, veillez à conserver le code Python `{request.remote_addr}` intégré dans le fichier `index.html` car il s'agit de l'aspect dynamique de l'exemple d'application Web.

```
cat templates/index.html
```

```
<html>
<head>
  <title>Exemple d'application</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <h1>You are calling me from {{ request.remote_addr }}</h1>
</body>
</html>

devasc@labvm:~/labs/devnet-src/sample-app$ cat static/style.css
body {background: lightsteelblue;}
devasc@labvm:~/labs/devnet-src/sample-app$
```

Étape 2: Mettez à jour le code Python de l'exemple d'application Web.

Maintenant que vous avez exploré les fichiers de base du site Web, vous devez mettre à jour le fichier **sample_app.py** afin qu'il affiche le fichier **index.html** au lieu de simplement renvoyer les données. La génération de contenu HTML à l'aide du code Python peut s'avérer lourde, en particulier lors de l'utilisation d'instructions conditionnelles ou de structures répétitives. Le fichier HTML peut être rendu automatiquement dans Flask à l'aide de la fonction `render_template`. Cela nécessite l'importation de la méthode **`render_template`** à partir de la bibliothèque flask et l'édition dans la fonction **`return`**. Apportez les modifications mises en surbrillance à votre script.

```
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

@sample.route("/")
def main():
    return render_template("index.html")

if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
```

Étape 3: Enregistrez et exécutez votre script.

Enregistrez et exécutez votre script **sample_app.py**. Vous devriez obtenir une sortie comme suit :

```
devasc@labvm:~/labs/devnet-src/sample-app$ python3 sample_app.py
* Serving Flask app "sample-app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

Remarque : Si vous avez obtenu la sortie Traceback et une erreur avec le message avec quelque chose comme **OserError : [Errno 98] Adresse déjà utilisée**, alors vous n'avez pas arrêté votre serveur précédent. Retournez à la fenêtre du terminal où ce serveur est en cours d'exécution et appuyez sur CTRL+C pour terminer le processus du serveur. Réexécutez votre script.

Étape 4: Vérifiez que votre programme est en cours d'exécution.

Encore une fois, vous pouvez vérifier que votre programme est exécuté de deux façons.

- Ouvrez le navigateur Web Chromium et entrez 0.0.0.0:8080 dans le champ URL. Vous devriez obtenir la même sortie qu'avant. Cependant, votre arrière-plan sera bleu acier clair et le texte sera formaté en H1.

You are calling me from 127.0.0.1

- Ouvrez une autre fenêtre de terminal et utilisez la commande **curl** pour vérifier la réponse du serveur. C'est là que vous verrez le résultat du code HTML rendu automatiquement à l'aide de la fonction `render_template`. Dans ce cas, vous obtiendrez tout le contenu HTML. Toutefois, le code Python dynamique sera remplacé par la valeur de `{{request.remote_addr}}`. Notez également que votre invite sera sur la même ligne que la dernière ligne de sortie HTML. Appuyez sur ENTRÉE pour obtenir une nouvelle ligne.

```
curl http://0.0.0.0:8080
```

Étape 5: Arrêtez le serveur.

Retournez à la fenêtre du terminal où le serveur est en cours d'exécution et appuyez sur CTRL+C pour arrêter le serveur.

Partie 5 : Créer un script Bash pour créer et exécuter un conteneur Docker

Une application peut être déployée sur un serveur bare metal (serveur physique dédié à un environnement à locataire unique) ou sur une machine virtuelle, comme vous venez de le faire dans la partie précédente. Il peut également être déployé dans une solution conteneurisée comme Docker. Dans cette partie, vous allez créer un script bash et y ajouter des commandes qui effectuent les tâches suivantes pour créer et exécuter un conteneur Docker :

- Créez des répertoires temporaires pour stocker les fichiers du site Web.
- Copiez les répertoires du site Web et `sample_app.py` dans le répertoire temporaire.
- Construire un Dockerfile.
- Build the Docker container.
- Démarrez le conteneur et vérifiez qu'il est en cours d'exécution.

Étape 1: Créez des répertoires temporaires pour stocker les fichiers du site Web.

Ouvrez le fichier de script **sample-app.sh** bash dans le répertoire `~/labs/devnet-src/sample-app`. Ajoutez le 'she-bang' et les commandes pour créer une structure de répertoire avec **tempdir** comme dossier parent.

```
#!/bin/bash

mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static
```

Étape 2: Copiez les répertoires du site Web et `sample_app.py` dans le répertoire temporaire.

dans le fichier **sample-app.sh**, ajoutez les commandes pour copier le répertoire du site Web et le script dans **tempdir**.

```
cp sample_app.py tempdir/.
cp -r modèles /* tempdir/templates/.
cp -r statique/* tempdir/statique/.
```

Étape 3: Créez un Dockerfile.

Dans cette étape, vous entrez les commandes bash **echo** nécessaires dans le fichier **sample-app.sh** pour créer un Dockerfile dans le **tempdir**. Ce Dockerfile sera utilisé pour construire le conteneur.

- a. Vous devez exécuter Python dans le conteneur, alors ajoutez la commande Docker **FROM** pour installer Python dans le conteneur.

```
echo "FROM python" > tempdir/Dockerfile
```

- b. Votre script **sample_app.py** a besoin de Flask, donc ajoutez la commande Docker **RUN** pour installer Flask dans le conteneur.

```
echo "RUN pip install flask" >> tempdir/Dockerfile
```

- c. Votre conteneur aura besoin des dossiers du site Web et du script **sample_app.py** pour exécuter l'application. Ajoutez donc les commandes Docker **COPY** pour les ajouter à un répertoire dans le conteneur Docker. Dans cet exemple, vous allez créer **/home/myapp** en tant que répertoire parent dans le conteneur Docker. En plus de copier le fichier **sample_app.py** dans le fichier Dockerfile, vous copierez également le fichier **index.html** à partir du répertoire **templates** et le fichier **style.css** à partir du répertoire **statique**.

```
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
```

```
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
```

```
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
```

- d. Utilisez la commande Docker **EXPOSE** pour exposer le port 8080 à utiliser par le serveur Web.

```
echo "EXPOSE 8080" >> tempdir/Dockerfile
```

- e. Enfin, ajoutez la commande Docker **CMD** pour exécuter le script Python.

```
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile
```

Étape 4: Construire le conteneur Docker.

Ajoutez les commandes au fichier **sample-app.sh** pour basculer vers le répertoire **tempdir** et construire le conteneur Docker. La commande **docker build -t** vous permet de spécifier le nom du conteneur et la période de fin (.) indique que vous voulez que le conteneur soit construit dans le répertoire courant.

```
cd tempdir
```

```
docker build -t sampleapp.
```

Étape 5: Démarrez le conteneur et vérifiez qu'il est en cours d'exécution.

- a. Ajoutez la commande **docker run** au fichier **sample-app.sh** pour démarrer le conteneur.

```
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
```

- b. Ajoutez la commande **docker ps -a** pour afficher tous les conteneurs Docker en cours d'exécution. Cette commande sera la dernière exécutée par le script bash.

```
docker ps -a
```

Étape 6: Enregistrez votre script bash.

Partie 6 : Créer, exécuter et vérifier le conteneur Docker

Dans cette partie, vous exécuterez un script bash qui fera les répertoires, copiera les fichiers, créera un Dockerfile, construira le conteneur Docker, exécutera une instance du conteneur Docker et affichera la sortie de la commande **docker ps -a** affichant les détails du conteneur en cours d'exécution. Ensuite, vous allez enquêter sur le conteneur Docker, arrêter le conteneur de s'exécuter et supprimer le conteneur.

Remarque : Assurez-vous que vous avez arrêté tous les autres processus de serveur Web que vous avez toujours en cours d'exécution à partir des parties précédentes de ce laboratoire.

Étape 1: Exécutez le script bash.

Exécutez le script bash à partir de la ligne de commande. You should see output similar to the following. Après avoir créé les répertoires **tempdir**, le script exécute les commandes pour construire le conteneur Docker. Notez que l'étape 7/7 de la sortie exécute le fichier **sample_app.py** qui crée le serveur Web. Notez également l'ID du conteneur. Vous verrez cela dans l'invite de commande Docker plus tard dans le laboratoire.

```
bash ./sample-app.sh
```

Étape 2: Examinez le conteneur Docker en cours d'exécution et l'application Web.

- La création des répertoires **tempdir** n'est pas affichée dans la sortie du script. Vous pouvez ajouter des commandes **echo** pour imprimer les messages lorsqu'ils sont créés avec succès. Vous pouvez également vérifier qu'ils sont là avec la commande **ls**. N'oubliez pas que ce répertoire contient les fichiers et dossiers utilisés pour construire le conteneur et lancer l'application Web. Ce n'est pas le conteneur qui a été construit.

```
ls tempdir/
```

- Notez le Dockerfile créé par votre script bash. Ouvrez ce fichier pour voir à quoi il ressemble dans sa forme finale sans les commandes **echo**.

```
cat tempdir/Dockerfile
```

- La sortie de la commande **docker ps -a** peut être difficile à lire en fonction de la largeur de l'affichage de votre terminal. Vous pouvez le rediriger vers un fichier texte où vous pouvez l'afficher mieux sans habillage de mots.

```
docker ps -a >> running.txt
```

- Le conteneur Docker crée sa propre adresse IP à partir d'un espace d'adressage réseau privé. Vérifiez que l'application Web est en cours d'exécution et signale l'adresse IP. Dans un navigateur Web à l'adresse **http://localhost:8080**, vous devriez voir le message **You are calling me from 172.17.0.1** formaté H1 sur un fond bleu acier clair. Vous pouvez également utiliser la commande **curl**, si vous le souhaitez.

```
curl http://172.17.0.1:8080
```

- Par défaut, Docker utilise le sous-réseau IPv4 172.17.0.0/16 pour la mise en réseau des conteneurs. (Cette adresse peut être modifiée si nécessaire.) Entrez la commande **ip address** pour afficher toutes les adresses IP utilisées par votre instance de la machine virtuelle DEVASC. Vous devriez voir l'adresse de bouclage 127.0.0.1 que l'application Web utilisée précédemment dans le laboratoire et la nouvelle interface Docker avec l'adresse IP 172.17.0.1.

```
ip address
```

Étape 3: Accédez et explorez le conteneur en cours d'exécution.

N'oubliez pas qu'un conteneur Docker est un moyen d'encapsuler tout ce dont vous avez besoin pour exécuter votre application afin qu'elle puisse être facilement déployée dans une variété d'environnements, et pas seulement dans votre machine virtuelle DEVASC.

- Pour accéder au conteneur en cours d'exécution, entrez la commande **docker exec -it** en spécifiant le nom du conteneur en cours d'exécution (samplerunning) et que vous voulez un shell bash (/bin/bash). L'option **-i** indique que vous voulez qu'elle soit interactive et l'option **-t** spécifie que vous voulez accéder au terminal. L'invite change à **root @containerID**. Votre identifiant de conteneur sera différent de celui indiqué ci-dessous. Notez que l'ID du conteneur correspond à l'ID affiché dans la sortie du **docker ps -a**.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker exec -it samplerunning /bin/bash
```

- b. Vous êtes maintenant en accès root pour le conteneur Docker **samplerunning**. À partir de là, vous pouvez utiliser les commandes Linux pour explorer le conteneur Docker. Saisissez **ls** pour voir la structure du répertoire au niveau racine.
- c. Rappelez-vous que dans votre script bash, vous avez ajouté des commandes dans le fichier Dockerfile qui ont copié vos répertoires et fichiers d'application dans le répertoire **home/myapp**. Entrez à nouveau la commande **ls** pour ce dossier afin d'afficher votre script **sample_app.py** et vos répertoires. Pour mieux comprendre ce qui est inclus dans votre conteneur Docker, vous pouvez utiliser la commande **ls** pour examiner d'autres répertoires tels que **/etc** et **/bin**.

```
ls home/myapp/
```

- d. Quittez le conteneur Docker pour revenir à la ligne de commande VM DEVASC.

Étape 4: Arrêtez et retirez le conteneur Docker.

- a. Vous pouvez arrêter le conteneur Docker avec la commande **docker stop** spécifiant le nom du conteneur en cours d'exécution. Il faudra quelques secondes pour nettoyer et mettre en cache le conteneur. Vous pouvez voir qu'il existe toujours en entrant la commande **docker ps -a**. Toutefois, si vous actualisez la page Web pour **http://localhost:8080**, vous verrez que l'application Web n'est plus en cours d'exécution.
- b. Vous pouvez redémarrer un conteneur arrêté avec la commande **docker start**. Le conteneur va immédiatement tourner vers le haut.
- c. Pour supprimer définitivement le conteneur, arrêtez-le d'abord, puis retirez-le avec la commande **docker rm**. Vous pouvez toujours le reconstruire en exécutant le programme **sample-app**. Utilisez la commande **docker ps -a** pour vérifier que le conteneur a été supprimé.